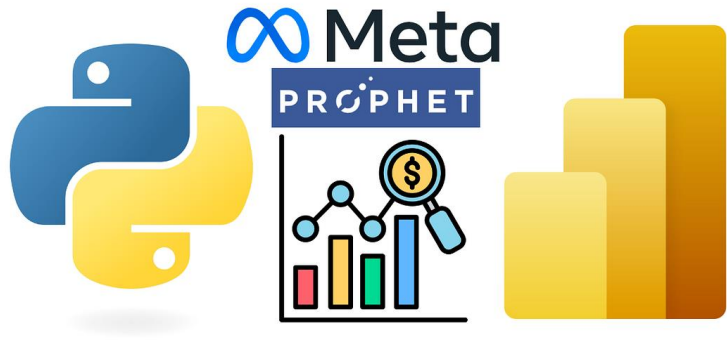
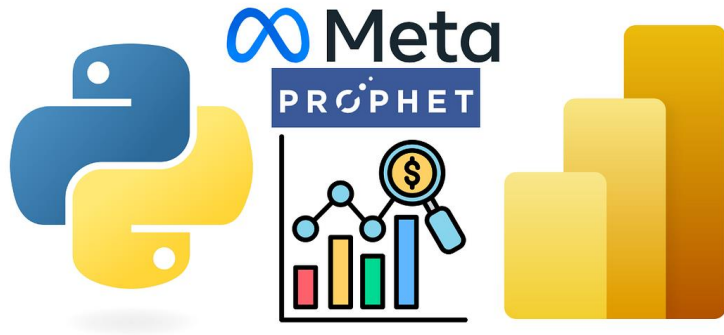




Prophet es una herramienta de código abierto desarrollada por **Meta (antes Facebook)** para la **predicción de series temporales**. Está diseñada para ser **fácil de usar**, incluso para personas sin experiencia profunda en estadística o modelado de series temporales, y es especialmente útil para datos con **tendencias estacionales y efectos recurrentes**.

https://facebook.github.io/prophet/docs/quick_start.html



Características principales de Prophet:

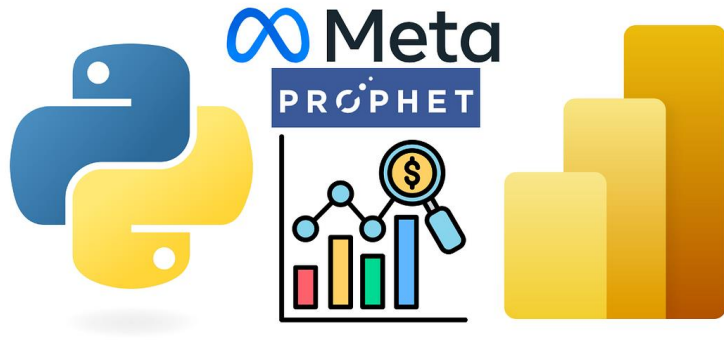
1. Modelo aditivo:

1. La serie temporal se modela como la suma de varios componentes:

$$y(t)=g(t)+s(t)+h(t)+\epsilon t \quad y(t)=g(t)+s(t)+h(t)+\epsilon t$$

1. Donde:

1. $g(t)$: tendencia (lineal o logística)
2. $s(t)$: estacionalidad (diaria, semanal, anual)
3. $h(t)$: efectos de días festivos o eventos especiales
4. ϵt : error aleatorio



Características principales de Prophet:

2. Manejo automático de estacionalidades:

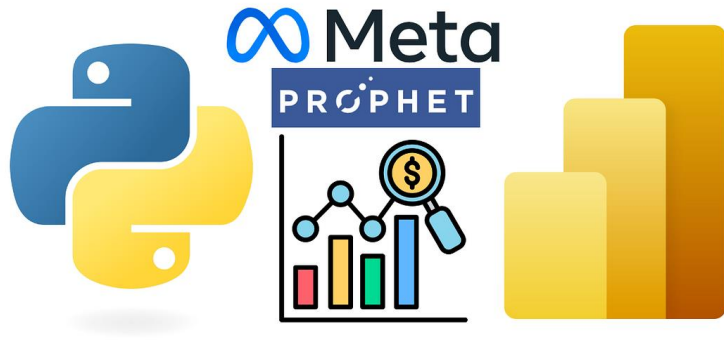
1. Detecta y ajusta patrones estacionales comunes.
2. Permite agregar estacionalidades personalizadas.

3. Soporte para días festivos:

1. Puedes incluir calendarios de días festivos específicos por país o personalizados.

4. Robustez ante datos faltantes y valores atípicos:

1. Prophet maneja bien series temporales con huecos o ruido.



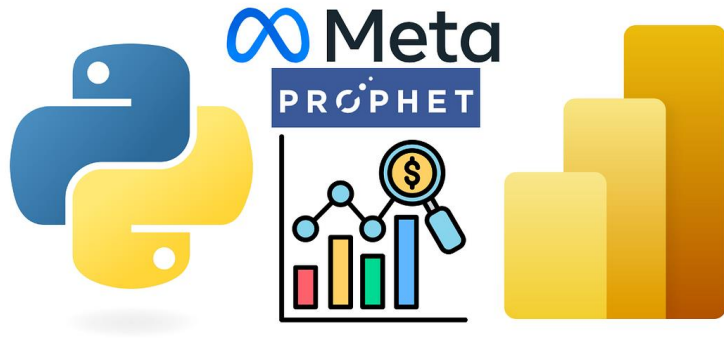
Características principales de Prophet:

5. Interfaz amigable:

- Disponible en **Python** y **R**.
- Requiere solo un DataFrame con dos columnas: `ds` (fecha) y `y` (valor).

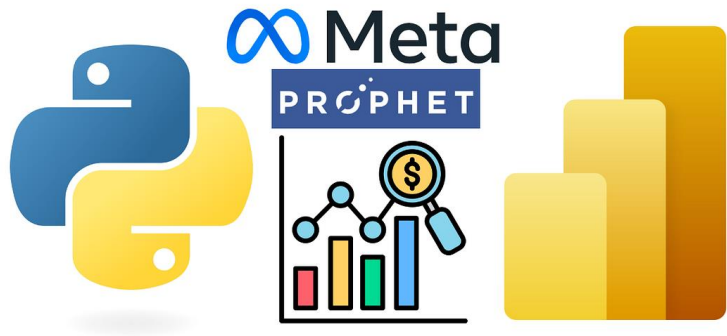
6. Predicciones con intervalos de confianza:

- Ofrece intervalos de incertidumbre para las predicciones.



Casos de uso comunes:

- Predicción de ventas o demanda.
- Análisis de tráfico web.
- Pronóstico de métricas financieras o de negocio.
- Planificación de capacidad o inventario.



```
import pandas as pd
from prophet import Prophet
```

```
df = pd.read_csv('data.csv')
df['ds'] = pd.to_datetime(df['date'])
df.head()
```

```
m = Prophet()
m.fit(df)
```

```
future = m.make_future_dataframe(periods=5)
future.tail()
```

Formato de entrada

ds	y
10/12/2007	9.590.761
11/12/2007	8.519.590
12/12/2007	8.183.677
13/12/2007	8.072.467
14/12/2007	7.893.572

31 Periodos a predecir

	DS
3265	2017-01-15
3266	2017-01-16
3267	2017-01-17
3268	2017-01-18
3269	2017-01-19

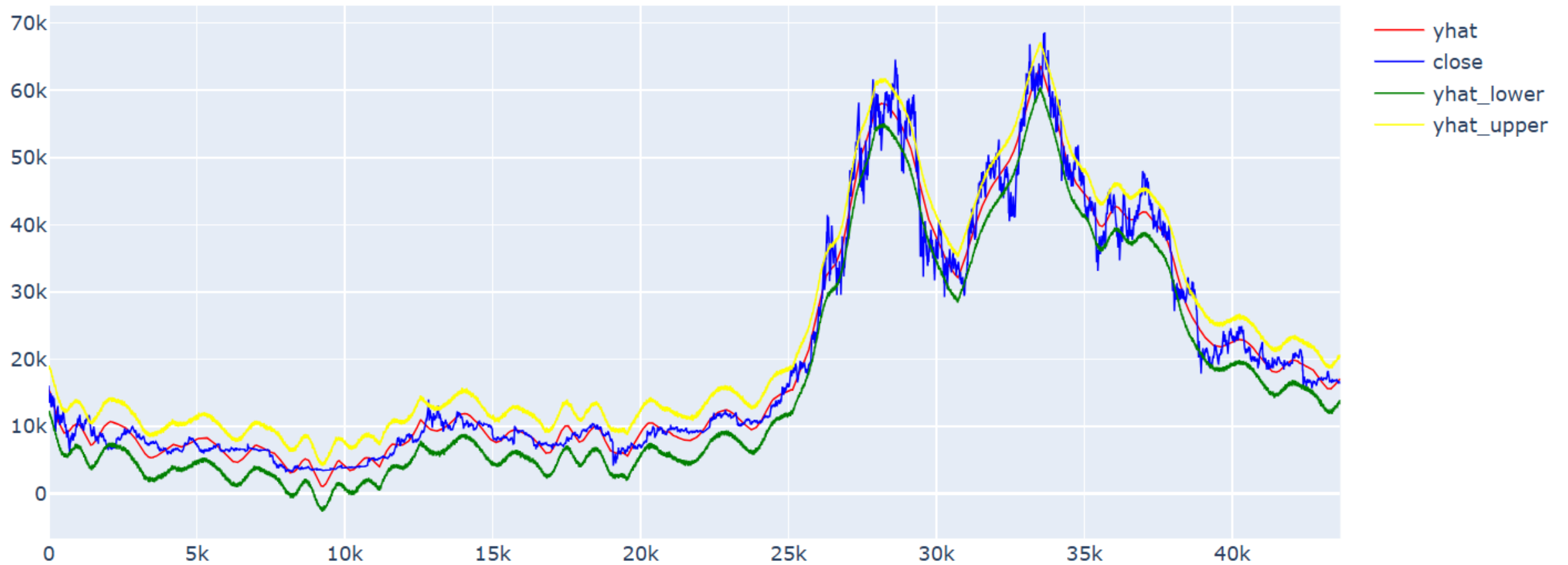


```
forecast = m.predict(future)
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail()
```

Predicciones

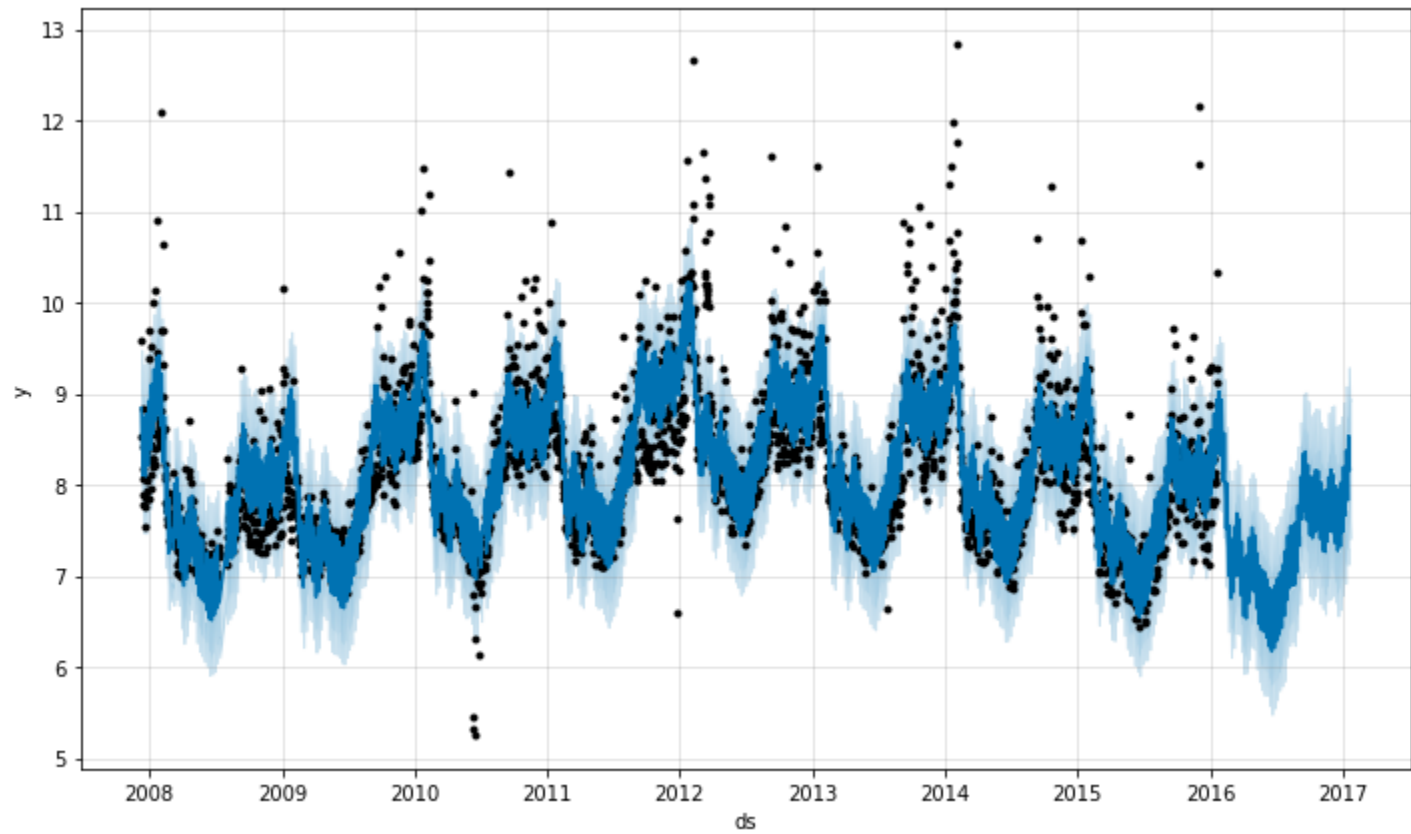
	DS	YHAT	YHAT_LOWER	YHAT_UPPER
3265	2017-01-15	8.212625	7.456310	8.959726
3266	2017-01-16	8.537635	7.842986	9.290934
3267	2017-01-17	8.325071	7.600879	9.072006
3268	2017-01-18	8.157723	7.512052	8.924022
3269	2017-01-19	8.169677	7.412473	8.946977

Columna	Significado
yhat	Es la predicción central del modelo para un punto en el tiempo. Representa el valor estimado de la variable objetivo (y) en esa fecha.
yhat_lower	Es el límite inferior del intervalo de confianza para la predicción. Indica el valor más bajo que se espera con una cierta probabilidad (por defecto, 80%).
yhat_upper	Es el límite superior del intervalo de confianza . Representa el valor más alto que se espera con esa misma probabilidad.



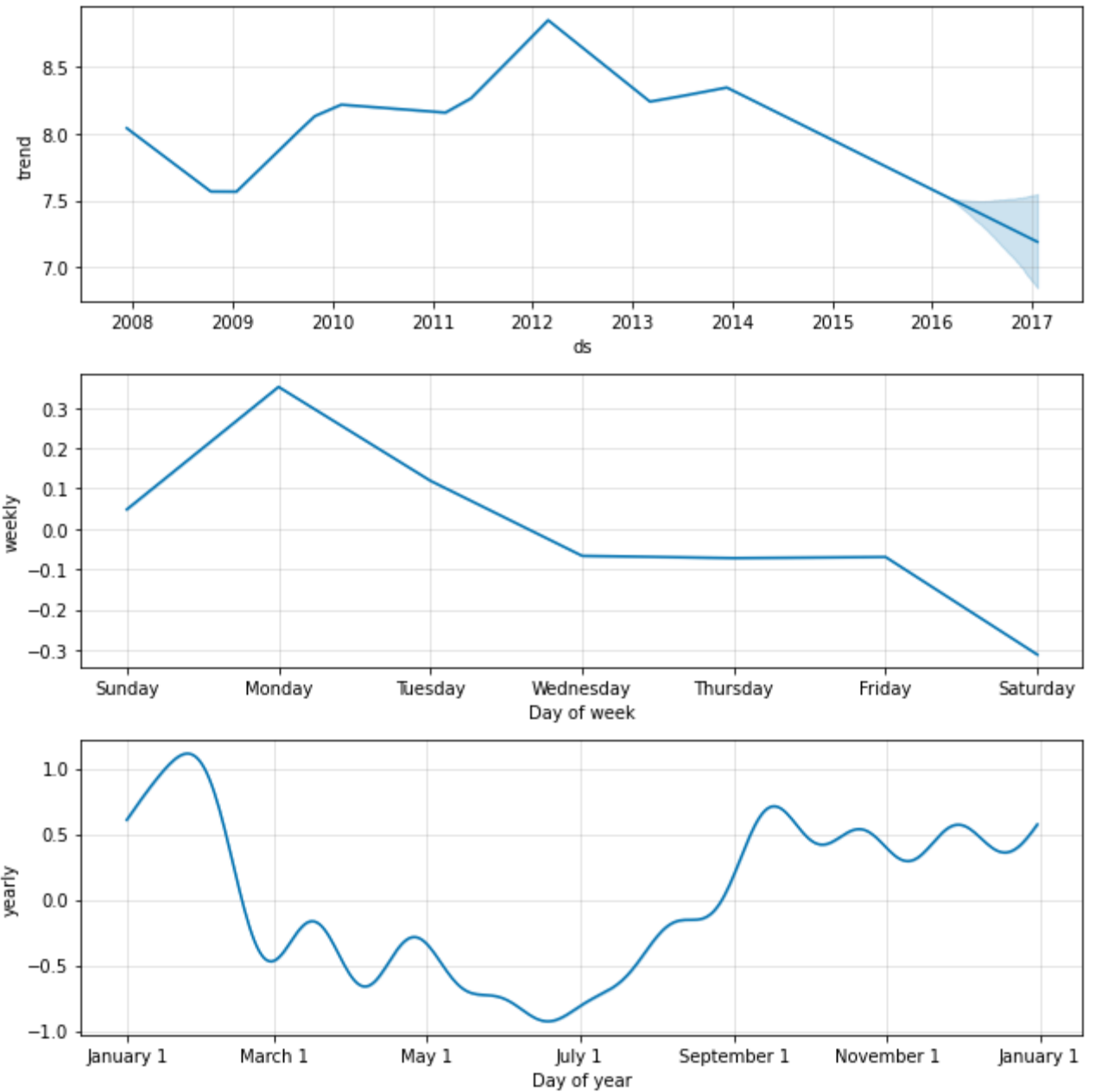


`m.plot(forecast)`





```
m.plot_components(forecast)
```





La clase **Prophet()** acepta varios parámetros que permiten personalizar el modelo de predicción.

⚙️ Parámetros principales de Prophet()

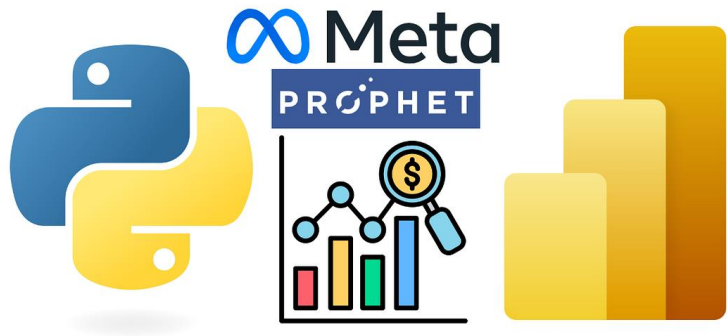
Parámetro	Tipo	Descripción
growth	str	Tipo de crecimiento: 'linear' (por defecto) o 'logistic' .
changepoints	list	Lista de fechas donde se permiten cambios en la tendencia. Si no se especifica, se detectan automáticamente.
n_changepoints	int	Número de puntos de cambio a detectar automáticamente (por defecto: 25).
changepoint_range	float	Proporción del historial donde se permiten cambios (por defecto: 0.8).
yearly_seasonality	bool o int	Habilita/deshabilita estacionalidad anual o define el número de Fourier terms.
weekly_seasonality	bool o int	Igual que el anterior, pero para estacionalidad semanal.
daily_seasonality	bool o int	Igual que el anterior, pero para estacionalidad diaria.
seasonality_mode	str	'additive' (por defecto) o 'multiplicative' .
seasonality_prior_scale	float	Controla la flexibilidad de la estacionalidad (por defecto: 10.0).
changepoint_prior_scale	float	Controla la flexibilidad de los puntos de cambio (por defecto: 0.05).
holidays	DataFrame	Permite incluir días festivos personalizados.
holidays_prior_scale	float	Controla la influencia de los días festivos (por defecto: 10.0).
interval_width	float	Ancho del intervalo de confianza (por defecto: 0.80).
uncertainty_samples	int	Número de muestras para estimar la incertidumbre (por defecto: 1000).
mcmc_samples	int	Número de muestras MCMC para inferencia bayesiana (por defecto: 0, es decir, no se usa).



⚙️ Parámetros principales de `make_future_dataframe()`

La función **`make_future_dataframe()`** acepta varios parámetros que permiten personalizar el modelo de predicción.

Parámetro	Tipo	Descripción
<code>periods</code>	<code>int</code>	Número de períodos futuros a predecir.
<code>freq</code>	<code>str</code>	Frecuencia de los datos (por defecto <code>'D'</code> para diario). Puedes usar <code>'H'</code> , <code>'W'</code> , <code>'M'</code> , etc.
<code>include_history</code>	<code>bool</code>	Si <code>True</code> (por defecto), incluye también las fechas históricas en el DataFrame resultante.



 **Hands on**

Realiza en tu kernel local de Python o en Google Colab la siguiente práctica:
(!pip install prophet o conda install -c conda-forge prophet)

1. Descarga datos de algún activo financiero usando la librería Yfinance

<https://www.geeksforgeeks.org/how-to-use-yfinance-api-with-python/>

2. Haz una predicción futura del campo 'Close' para los próximos periodos. ¿Va a subir el activo? Compramos ? 😊
3. Realiza los plots qué conoces.

```
import yfinance as yf
import matplotlib.pyplot as plt

# Descargar datos históricos de BTC-USD en temporalidad horaria (últimos 5 días)
btc_data = yf.download(tickers='BTC-USD', period='5d', interval='1h')

# Mostrar los primeros registros
print(btc_data.head())
```